

Fixed Income Analytics Platform

Mathematical Foundations and Worked Examples

Jun Shen

Abstract

This report lays out the mathematics behind every quantitative component of the Fixed Income Analytics Platform: bond cashflow mechanics, the bond pricing formula, accrued interest, yield-to-maturity solving, four yield curve interpolation methods, duration and convexity, key rate duration, curve shock scenarios, and portfolio-level risk aggregation. Each section derives the relevant formula from first principles and then walks through a small numeric example that matches, line for line, what the corresponding Python code in `src/` actually computes – so a reader can check the math by hand and then find the exact same numbers coming out of the code.

Contents

1	Scope and Notation	1
2	Coupon Schedule Construction	1
3	Bond Pricing	2
3.1	Full-period pricing	2
3.2	Fractional-period pricing (settlement between coupon dates)	2
4	Accrued Interest and Clean vs. Dirty Price	3
5	Yield to Maturity: The Inverse Problem	3
6	Yield Curve Interpolation	4
6.1	Linear interpolation	4
6.2	Cubic spline interpolation	4
6.3	Nelson-Siegel model	5
6.4	Nelson-Siegel-Svensson (NSS) extension	6
7	Duration	6
7.1	Macaulay duration	6
7.2	Modified duration	6
8	DV01 (Dollar Duration)	7
9	Convexity	7
10	Key Rate Duration and KR01	8
10.1	Discounting off the curve	8
10.2	Triangular key rate shocks	9

11 Yield Curve Shocks	9
11.1 Parallel shocks	9
11.2 Non-parallel shocks	10
11.3 P&L attribution	10
12 Portfolio Aggregation	10
13 Modeling Simplifications and Limitations	11

1 Scope and Notation

This platform prices plain-vanilla, fixed-coupon U.S. Treasury securities (bills, notes, and bonds) and builds risk analytics on top of that pricing engine. Throughout this report:

- F – face (par) value of the bond, quoted per 100.
- c – annual coupon rate, as a decimal (e.g. 0.0425 for 4.25%).
- m – number of coupon payments per year ($m = 2$ for semiannual Treasuries).
- $C = F \cdot c/m$ – the dollar amount of a single coupon payment.
- y – annualized yield to maturity, compounded m times per year, as a decimal.
- t – time, generally measured in years from the settlement date.
- $P_{\text{dirty}}, P_{\text{clean}}$ – dirty (full) price and clean (quoted) price.

All dollar amounts are quoted per 100 of face value unless stated otherwise, which is the standard bond market convention. The corresponding source file for each section is named explicitly so the math can be checked directly against the implementation.

2 Coupon Schedule Construction

Corresponds to: `src/bond.py, Bond.generate_coupon_dates()`.

A fixed-coupon bond with maturity date T_{mat} and m payments per year pays coupons every $12/m$ months. The coupon schedule is built by walking backward from T_{mat} in steps of $12/m$ months until the walk reaches or passes the issue date T_{issue} :

$$T_k = T_{\text{mat}} - k \cdot \frac{12}{m} \text{ months}, \quad k = 0, 1, 2, \dots$$

stopping at the largest k such that $T_k > T_{\text{issue}}$. The resulting dates are reversed into chronological order to give the full coupon schedule $\{T_1, T_2, \dots, T_n = T_{\text{mat}}\}$.

Worked example. Take a 2-year, 5% semiannual bond issued on 2023-03-15 and maturing 2025-03-15 (`Bond("EX_2Y", 100, 0.05, 2, date(2023,3,15), date(2025,3,15))`). Walking backward in 6-month steps from 2025-03-15 gives:

$$\{2023-09-15, 2024-03-15, 2024-09-15, 2025-03-15\}$$

which is exactly what `generate_coupon_dates()` returns for this bond.

3 Bond Pricing

Corresponds to: `src/pricing_engine.py, PricingEngine.price_from_yield()`.

3.1 Full-period pricing

For a bond priced exactly on a coupon date, the standard discounted-cashflow formula applies directly. If the bond has n remaining coupons of size C plus a final principal repayment F , and the discount rate per period is y/m , the dirty price is

$$P_{\text{dirty}} = \sum_{i=1}^n \frac{C}{(1 + y/m)^i} + \frac{F}{(1 + y/m)^n}. \quad (1)$$

3.2 Fractional-period pricing (settlement between coupon dates)

In practice, settlement almost never lands exactly on a coupon date. Let w be the fraction of the *current* coupon period that remains between settlement and the next coupon payment:

$$w = \frac{\text{days from settlement to next coupon}}{\text{days in the current coupon period}}. \quad (2)$$

The remaining coupon dates are indexed $i = 1, \dots, n$ starting with the next coupon. Each cashflow CF_i (equal to C , or $C + F$ for the final payment) is discounted back by $i - 1 + w$ periods instead of a whole number of periods:

$$P_{\text{dirty}} = \sum_{i=1}^n \frac{CF_i}{(1 + y/m)^{i-1+w}}. \quad (3)$$

This is the fractional-period (or "Street") pricing convention, and it is exactly what `PricingEngine.price_f` implements: the code computes `w` from the previous and next coupon dates, then loops over the remaining cashflows accumulating `cash / (1 + ytm/m) ^ (i + w)` for `i = 0, 1, 2, ...` (using 0-based indexing, which is equivalent to $i - 1$ in the 1-based formula above).

Clean price is simply

$$P_{\text{clean}} = P_{\text{dirty}} - AI, \quad (4)$$

where AI is accrued interest, derived in Section 4.

Worked example. Continue with the 2-year 5% bond from Section 2, settled on 2024-06-15. The remaining cashflows are

$$(2024-09-15, 2.5), \quad (2025-03-15, 102.5)$$

The current coupon period runs from 2024-03-15 to 2024-09-15 (184 days); settlement on 2024-06-15 is 92 days into that period, so

$$w = \frac{184 - 92}{184} = \frac{92}{184} = 0.5.$$

Pricing at $y = 4.5\%$:

$$P_{\text{dirty}} = \frac{2.5}{(1.0225)^{0.5}} + \frac{102.5}{(1.0225)^{1.5}} = 101.607770.$$

This matches `price_from_yield(0.045, date(2024,6,15))`, which returns `dirty_price = 101.60776989830813`.

4 Accrued Interest and Clean vs. Dirty Price

Corresponds to: `src/bond.py`, `Bond.accrued_interest()`.

Accrued interest compensates the seller of a bond for the coupon income earned between the last coupon date and settlement. Using the fraction-of-period method (Actual/Actual):

$$AI = C \times \frac{\text{days since previous coupon}}{\text{days in current coupon period}}. \quad (5)$$

Worked example. Same bond and settlement date as above (2024-06-15). The previous coupon was 2024-03-15, the next is 2024-09-15 (184 days apart), and settlement is 92 days after the previous coupon:

$$AI = 2.5 \times \frac{92}{184} = 1.25.$$

The code returns exactly `accrued_interest = 1.25` for this settlement date, and

$$P_{\text{clean}} = 101.607770 - 1.25 = 100.357770,$$

which matches `clean_price = 100.35776989830813` from the pricing engine.

5 Yield to Maturity: The Inverse Problem

Corresponds to: `src/yield_solver.py`, `YieldSolver.solve_yield()`.

Given a market price, yield to maturity is the single discount rate y that satisfies Equation 3 when P_{dirty} (or equivalently P_{clean} plus known accrued interest) is set equal to the observed market price. There is no closed-form inverse of Equation 3 in y , so it has to be solved numerically.

Bond price is a smooth, strictly decreasing function of yield over any economically sensible range, which makes **bisection** a safe and simple choice: given a bracket $[y_{\text{lo}}, y_{\text{hi}}]$ where $P(y_{\text{lo}}) \geq P_{\text{target}} \geq P(y_{\text{hi}})$, evaluate the midpoint, discard whichever half of the bracket doesn't contain the root, and repeat until the price error is below tolerance:

$$y_{k+1} = \frac{y_{\text{lo},k} + y_{\text{hi},k}}{2}, \quad \begin{cases} y_{\text{lo},k+1} = y_{k+1} & \text{if } P(y_{k+1}) > P_{\text{target}} \\ y_{\text{hi},k+1} = y_{k+1} & \text{otherwise} \end{cases} \quad (6)$$

This converges linearly (roughly one extra correct bit of precision per iteration) but never diverges and never needs a derivative – a reasonable trade against Newton-Raphson, which converges faster but can misbehave without a good starting point.

Worked example. Take the clean price computed above, $P_{\text{clean}} = 100.357770$, and solve for the yield that reproduces it. `YieldSolver.solve_yield()` returns $y = 0.044999999925 \dots \approx 4.5\%$, recovering the original yield used to price the bond to eight decimal places of tolerance – confirming the pricing engine and the solver are consistent inverses of each other.

6 Yield Curve Interpolation

Corresponds to: `src/interpolation.py`.

A market Treasury curve only quotes yields at a handful of tenors (e.g. 3M, 1Y, 2Y, 5Y, 10Y, 30Y). Pricing or risk-managing a bond that matures at, say, 15 years requires estimating a yield at a maturity that isn't directly quoted. Four methods are implemented.

6.1 Linear interpolation

The simplest approach: given two bracketing curve points (t_0, y_0) and (t_1, y_1) with $t_0 \leq t \leq t_1$,

$$y(t) = y_0 + \frac{t - t_0}{t_1 - t_0}(y_1 - y_0). \quad (7)$$

Outside the quoted range, the curve is extrapolated flat (the nearest endpoint's yield is used).

Worked example. With curve points $(1, 3.0\%)$, $(2, 3.5\%)$, $(5, 4.0\%)$, $(10, 4.5\%)$, interpolating at $t = 3$ falls between the $(2, 3.5\%)$ and $(5, 4.0\%)$ points:

$$y(3) = 3.5\% + \frac{3 - 2}{5 - 2}(4.0\% - 3.5\%) = 3.5\% + \frac{1}{3}(0.5\%) = 3.6\bar{6}\%.$$

`LinearInterpolator.interpolate(3)` returns `0.036666...`, matching exactly.

6.2 Cubic spline interpolation

A natural cubic spline fits a separate cubic polynomial

$$S_j(t) = a_j + b_j(t - t_j) + c_j(t - t_j)^2 + d_j(t - t_j)^3 \quad (8)$$

on each interval $[t_j, t_{j+1}]$, chosen so that the spline, its first derivative, and its second derivative are all continuous across interior knots, and the second derivative is pinned to zero at both endpoints (the "natural" boundary condition). This is the standard construction from numerical analysis (see Burden & Faires, *Numerical Analysis*), solved as follows.

Let $h_j = t_{j+1} - t_j$. The interior second-derivative terms c_j satisfy the tridiagonal system

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_jc_{j+1} = 3 \left(\frac{y_{j+1} - y_j}{h_j} - \frac{y_j - y_{j-1}}{h_{j-1}} \right), \quad (9)$$

with $c_0 = c_n = 0$ from the natural boundary condition. This is solved efficiently with the Thomas algorithm (forward elimination, then back-substitution) rather than a general dense solver, since the system is tridiagonal. Once every c_j is known, the remaining coefficients follow directly:

$$a_j = y_j, \quad (10)$$

$$b_j = \frac{y_{j+1} - y_j}{h_j} - \frac{h_j(c_{j+1} + 2c_j)}{3}, \quad (11)$$

$$d_j = \frac{c_{j+1} - c_j}{3h_j}. \quad (12)$$

`CubicSplineInterpolator._build_spline_coefficients()` implements exactly this forward-elimination/back-substitution sequence with explicit loops – it does not call a general-purpose linear algebra routine.

Worked example. Using the same four curve points as the linear example, $(1, 3.0\%)$, $(2, 3.5\%)$, $(5, 4.0\%)$, $(10, 4.5\%)$, the spline solves to:

Segment j	a_j	b_j	c_j
[1, 2]	0.030000	0.005431	0.000000
[2, 5]	0.035000	0.004137	-0.001294
[5, 10]	0.040000	0.000608	0.000118

(with d_j values -0.000431 , 0.000157 , -0.0000078 respectively). Evaluating at $t = 3$ (inside segment $[2, 5]$, so $\Delta t = 1$):

$$S(3) = 0.035000 + 0.004137(1) - 0.001294(1)^2 - 0.000157(1)^3 \approx 0.038000$$

which matches `CubicSplineInterpolator.interpolate(3) = 0.038000`. Note this differs from the linear estimate of 0.036667 – the spline lets the curvature already visible in the first three points (yield increases are decelerating, then reaccelerate) bend the interpolated value upward slightly rather than following a straight line.

6.3 Nelson-Siegel model

The Nelson-Siegel model represents the whole curve with three interpretable factors: a long-run level β_0 , a short-term slope β_1 , and a curvature/hump term β_2 , governed by a single decay parameter τ :

$$y(t) = \beta_0 + \beta_1 \left(\frac{1 - e^{-t/\tau}}{t/\tau} \right) + \beta_2 \left(\frac{1 - e^{-t/\tau}}{t/\tau} - e^{-t/\tau} \right). \quad (13)$$

Write $L(t; \tau) = \frac{1 - e^{-t/\tau}}{t/\tau}$ (the "slope loading", which decays from 1 at $t = 0$ toward 0 as $t \rightarrow \infty$) and $C(t; \tau) = L(t; \tau) - e^{-t/\tau}$ (the "curvature loading", which is zero at $t = 0$ and $t \rightarrow \infty$ and peaks at an intermediate maturity controlled by τ). As $t \rightarrow \infty$, $y(t) \rightarrow \beta_0$, so β_0 is naturally interpreted as the long-run level of the curve; as $t \rightarrow 0$, $y(t) \rightarrow \beta_0 + \beta_1$, so β_1 controls the short-end slope relative to the long end.

Calibration without a nonlinear optimizer. For a *fixed* τ , $y(t)$ is linear in $\beta_0, \beta_1, \beta_2$ – it's an ordinary linear regression with design matrix columns $[1, L(t_i; \tau), C(t_i; \tau)]$. This is the approach Diebold and Li (2006) use in practice: rather than running a general nonlinear least-squares solver over all four parameters at once, grid-search over candidate τ values, and at *each* grid point solve the β 's exactly by ordinary least squares (OLS):

$$\hat{\beta}(\tau) = (X(\tau)^\top X(\tau))^{-1} X(\tau)^\top \mathbf{y} \quad (14)$$

then keep whichever τ (and its associated $\hat{\beta}$) minimizes the sum of squared residuals. `NelsonSiegelModel.calibrate()` implements exactly this: it sweeps τ from 0.25 to 30.0 in steps of 0.25, builds the 3×3 normal equations $X^\top X \beta = X^\top \mathbf{y}$ by hand at each candidate τ , solves them with a small Gaussian-elimination routine (`_solve_linear_system`, using partial pivoting), computes the resulting sum of squared errors, and keeps the best. No scipy or numpy optimizer is used anywhere in this calibration.

Worked example. Calibrating against the ten sample curve points in `data/treasury_curve.csv` (0.25Y through 30Y) gives

$$\hat{\beta}_0 = 0.046077, \quad \hat{\beta}_1 = -0.002066, \quad \hat{\beta}_2 = -0.018631, \quad \hat{\tau} = 2.25,$$

with a residual sum of squared errors of about 9.44×10^{-7} across the ten points – i.e. the fit is accurate to a small fraction of a basis point on average. Evaluating the fitted curve at $t = 15$ (not one of the ten input points) gives $y(15) = 4.29998\%$, close to both the linear estimate (4.30%, since 15 sits almost exactly between the 10Y and 20Y quotes) and the cubic spline estimate for the same point.

6.4 Nelson-Siegel-Svensson (NSS) extension

The NSS model adds a second curvature term with its own decay parameter τ_2 , which lets the fitted curve display two humps instead of one – useful when a real curve has more structure than the 3-factor NS model can represent (for example, a pronounced hump around 5-7 years combined with a separate long-end hump around 20 years):

$$y(t) = \beta_0 + \beta_1 L(t; \tau_1) + \beta_2 C(t; \tau_1) + \beta_3 C(t; \tau_2). \quad (15)$$

Just as with NS, for fixed (τ_1, τ_2) this is linear in $\beta_0, \dots, \beta_3$, so `NelsonSiegelSvenssonModel.calibrate()` grid-searches over a 2D (τ_1, τ_2) grid (skipping pairs where $\tau_1 \approx \tau_2$, since the two curvature columns become nearly collinear and the 4×4 regression turns numerically unstable), solving a 4×4 OLS system at every grid point with the same hand-written Gaussian elimination routine used for NS.

7 Duration

Corresponds to: `src/risk_measures.py`, `RiskMeasures.macauley_duration()` and `modified_duration()`.

7.1 Macauley duration

Macauley duration is the cashflow-weighted average time to receipt of a bond's cashflows, with each cashflow's present value used as the weight:

$$D_{\text{Mac}} = \frac{\sum_i t_i \cdot PV(CF_i)}{\sum_i PV(CF_i)} = \frac{\sum_i t_i \cdot \frac{CF_i}{(1 + y/m)^{t_i m}}}{P_{\text{dirty}}}. \quad (16)$$

Here t_i is measured in years from settlement to cashflow i . This platform uses the standard whole-years-from-settlement convention for duration (rather than the exact fractional-period exponent used for pricing in Equation 3) – this is a common textbook simplification for duration specifically, and Section 11 notes where it can diverge slightly from the exact pricing formula.

7.2 Modified duration

Modified duration adjusts Macauley duration for the compounding frequency, and is the first-order (linear) sensitivity of price to yield:

$$D_{\text{Mod}} = \frac{D_{\text{Mac}}}{1 + y/m}, \quad \frac{1}{P} \frac{\partial P}{\partial y} \approx -D_{\text{Mod}}. \quad (17)$$

The derivation follows directly from differentiating Equation 1 with respect to y :

$$\begin{aligned} \frac{\partial P}{\partial y} &= \sum_i \frac{\partial}{\partial y} [CF_i (1 + y/m)^{-t_i m}] = -\frac{1}{1 + y/m} \sum_i t_i \cdot CF_i (1 + y/m)^{-t_i m} \\ &= -\frac{1}{1 + y/m} \sum_i t_i \cdot PV(CF_i) = -\frac{D_{\text{Mac}} \cdot P}{1 + y/m} = -D_{\text{Mod}} \cdot P, \end{aligned} \quad (18)$$

so $\partial P / \partial y \approx -D_{\text{Mod}} \cdot P$, or equivalently $\% \Delta P \approx -D_{\text{Mod}} \cdot \Delta y$.

Worked example. A 10-year, 4.25% semiannual bond (UST_10Y in the sample data, issued 2022-01-15, maturing 2032-01-15), priced at settlement 2025-01-15 with $y = 4.15\%$:

$$P_{\text{dirty}} = 100.602130, \quad D_{\text{Mac}} = 6.129142, \quad D_{\text{Mod}} = \frac{6.129142}{1 + 0.0415/2} = 6.004548.$$

These are exactly the values returned by `RiskMeasures.macauley_duration()` and `modified_duration()` for this bond and settlement date.

8 DV01 (Dollar Duration)

Corresponds to: `src/risk_measures.py`, `RiskMeasures.dv01()`.

DV01 (dollar value of a basis point) is the dollar price change for a 1bp move in yield. Rather than deriving DV01 analytically from duration, this platform computes it directly by *central finite difference* on the pricing engine – bump the yield up and down by a small amount h (1bp, i.e. $h = 0.0001$) and reprice:

$$DV01 \approx \frac{P(y - h) - P(y + h)}{2}. \quad (19)$$

This is deliberately independent of the duration formula in Section 7 – computing it this way, directly off the pricing engine, gives a useful cross-check: DV01 should be very close to $D_{\text{Mod}} \cdot P \cdot 0.0001$, and any large discrepancy between the finite-difference DV01 and the analytical approximation is a signal that something is off (a mismatched settlement date, a badly scaled yield input, etc.) rather than expected behavior.

Worked example. For the same 10-year bond as above ($P_{\text{dirty}} = 100.602130$, $D_{\text{Mod}} = 6.004548$), the finite-difference DV01 comes out to 0.060424. The analytical approximation is

$$D_{\text{Mod}} \cdot P \cdot 0.0001 = 6.004548 \times 100.602130 \times 0.0001 = 0.060409,$$

which agrees with the finite-difference number to within about 0.02% – the small residual gap is exactly the convexity effect that a purely linear duration-based estimate misses, discussed next.

9 Convexity

Corresponds to: `src/risk_measures.py`, `RiskMeasures.convexity()`.

Duration is only the first-order (linear) term in a Taylor expansion of price around a change in yield. Convexity is the second-order term, and captures the fact that the price/yield relationship is curved (convex, for a plain vanilla bond), not straight:

$$\Delta P \approx \underbrace{-D_{\text{Mod}} \cdot P \cdot \Delta y}_{\text{duration term}} + \underbrace{\frac{1}{2} \text{Convexity} \cdot P \cdot (\Delta y)^2}_{\text{convexity term}}. \quad (20)$$

Equivalently, in percentage terms:

$$\frac{\Delta P}{P} \approx -D_{\text{Mod}} \cdot \Delta y + \frac{1}{2} \text{Convexity} \cdot (\Delta y)^2. \quad (21)$$

Analytically, convexity is $\frac{1}{P} \frac{\partial^2 P}{\partial y^2}$; this platform again computes it by finite difference rather than an analytical cashflow-weighted sum, using the standard central second-difference formula:

$$\text{Convexity} \approx \frac{P(y + h) + P(y - h) - 2P(y)}{P(y) \cdot h^2}. \quad (22)$$

Worked example. Same 10-year bond, $y = 4.15\%$, $P = 100.602130$, $D_{\text{Mod}} = 6.004548$, Convexity = 42.130010. Apply a large +100bp shock ($\Delta y = 0.01$) to see the convexity correction matter:

Quantity	Value
Actual repriced % change	−5.800885%
Duration-only approximation	−6.004548%
Duration + convexity approximation	−5.793898%

The duration-only estimate overshoots the actual price decline by about 20bp of price (it assumes the price/yield line is straight, so it overstates the loss as yields sell off). Adding the convexity term,

$$-6.004548\% + \frac{1}{2}(42.130010)(0.01)^2 \times 100 = -6.004548\% + 0.210650\% = -5.793898\%,$$

brings the estimate within about 7bp of the actual −5.800885% – convexity captures most, though not quite all, of the remaining gap (the residual comes from third- and higher-order terms in the Taylor expansion, which are small but not exactly zero for a 100bp move).

10 Key Rate Duration and KR01

Corresponds to: `src/risk_measures.py`, `KeyRateDuration`.

Duration and DV01 (Sections 7–8) measure sensitivity to a *parallel* move in a single flat yield. In reality, curves rarely move in parallel – the 2Y and the 30Y can move by very different amounts on the same day. Key rate duration (KRD) measures sensitivity to a bump at just *one* point on the curve, holding the rest of the curve fixed.

10.1 Discounting off the curve

To measure sensitivity to individual curve points, each bond cashflow must be discounted at a rate appropriate to its own maturity, rather than a single flat yield:

$$P = \sum_i \frac{CF_i}{(1 + r(t_i)/m)^{t_i m}}, \quad (23)$$

where $r(t_i)$ is the curve yield interpolated to cashflow i 's maturity (using whichever interpolator is passed in – linear, cubic spline, or one of the Nelson-Siegel variants). As noted in Section 13 below, this platform uses the loaded par curve directly as a proxy for the discount/spot curve rather than bootstrapping a true zero curve first.

10.2 Triangular key rate shocks

For each key rate tenor T_k in a chosen set $\{T_1, \dots, T_K\}$ (for example $\{0.5, 1, 2, 5, 10, 20, 30\}$ years), a **triangular shock** is applied: a bump of size h at T_k itself, tapering linearly to zero at the neighboring key rate tenors T_{k-1} and T_{k+1} , and left unchanged everywhere else:

$$\Delta r(t) = \begin{cases} h & t = T_k \\ h \cdot \frac{t - T_{k-1}}{T_k - T_{k-1}} & T_{k-1} < t < T_k \\ h \cdot \frac{T_{k+1} - t}{T_{k+1} - T_k} & T_k < t < T_{k+1} \\ 0 & \text{otherwise} \end{cases} \quad (24)$$

(with the obvious one-sided modification at the first and last key rate tenors, which are extended flat rather than tapering to zero). This is the standard triangular key-rate shock methodology used across fixed income risk systems. Key rate duration and key rate 01 are then

$$KRD(T_k) = \frac{P(r - h_k) - P(r + h_k)}{2h \cdot P(r)}, \quad KR01(T_k) = \frac{P(r - h_k) - P(r + h_k)}{2}, \quad (25)$$

where $r + h_k$ and $r - h_k$ denote the curve with the triangular shock in Equation 24 applied at $\pm h$ around key rate T_k .

Worked example. Take the same 10-year bond and a flat 4.0% curve quoted at $\{1, 2, 5, 10, 20, 30\}$ years. Computing KRD at each key rate tenor:

Key rate tenor	KRD	KR01
1Y	0.0443	0.00045
2Y	0.1950	0.00198
5Y	3.6244	0.03680
10Y	2.1494	0.02182
20Y	0.0000	0.00000
30Y	0.0000	0.00000

This makes intuitive sense: the bond's cashflows run out to 10 years, so bumps beyond the 10Y key rate node have essentially no effect on its price, while most of the exposure sits at the 5Y and 10Y nodes – the two key rate tenors that bracket most of the bond's remaining life (coupon cashflows are spread between years 0 and 10, and the triangular weighting concentrates around wherever the bulk of the present value sits, which for a 10-year bond is toward the back half of its life because of the large final principal payment). Summing KRD across all six tenors gives approximately 6.005, matching the bond's effective duration computed from a full parallel shock to the curve – confirming the key rate decomposition adds back up to the whole.

11 Yield Curve Shocks

Corresponds to: `src/shock_engine.py`, `ShockEngine`.

11.1 Parallel shocks

The simplest scenario: every tenor on the curve moves by the same number of basis points, $\Delta r(t) = h$ for all t .

11.2 Non-parallel shocks

Three shapes are implemented, each defined by a piecewise-linear shock function over maturity:

$$\text{Steepener/Flattener: } \Delta r(t) = h_{\text{short}} + \frac{t - t_{\min}}{t_{\max} - t_{\min}} (h_{\text{long}} - h_{\text{short}}) \quad (26)$$

$$\text{Twist (pivot } T_p): \Delta r(t) = \begin{cases} h_{\text{short}} \cdot \frac{T_p - t}{T_p - t_{\min}} & t \leq T_p \\ h_{\text{long}} \cdot \frac{t - T_p}{t_{\max} - T_p} & t > T_p \end{cases} \quad (27)$$

$$\text{Butterfly (belly } T_b): \Delta r(t) = \begin{cases} h_{\text{wing}} + \frac{t - t_{\min}}{T_b - t_{\min}} (h_{\text{belly}} - h_{\text{wing}}) & t \leq T_b \\ h_{\text{belly}} + \frac{t - T_b}{t_{\max} - T_b} (h_{\text{wing}} - h_{\text{belly}}) & t > T_b \end{cases} \quad (28)$$

A steepener uses $h_{\text{short}} < 0 < h_{\text{long}}$ (short end rallies, long end sells off, widening the spread); a flattener is the same mechanism with the signs reversed. A twist pivots the curve rotation around T_p (the yield at exactly T_p is unchanged by construction). A butterfly moves the belly of the curve opposite to the wings (short and long ends), tapering linearly in between.

Worked example. Applying a +50bp parallel shock to the sample curve in `data/treasury_curve.csv` shifts every quoted tenor up by exactly 0.50%, e.g. the 10Y point moves from 4.15% to 4.65% and the 2Y moves from 4.00% to 4.50% – every tenor by the identical 50bp, as expected for a pure parallel shift.

11.3 P&L attribution

For a single bond repriced under a flat-yield shock $y \rightarrow y + \Delta y$, `ShockEngine.evaluate_flat_yield_shock()` reports four numbers side by side: the actual dollar P&L and % price change from fully repricing the bond, and the duration-only and duration-plus-convexity Taylor approximations from Equations 20 and 21. This is the same comparison worked through numerically in Section 9.

12 Portfolio Aggregation

Corresponds to: `src/portfolio.py`, `Portfolio`.

Each holding is a bond plus a notional face amount held in dollars, N_j (not the same as the bond’s own `face_value`, which stays fixed at 100 for price quoting – N_j is the actual size of the position). A holding’s market value is

$$MV_j = \frac{P_{\text{dirty},j}}{100} \times N_j. \quad (29)$$

Portfolio market value is $MV = \sum_j MV_j$, and each holding’s weight is $w_j = MV_j/MV$. Portfolio-level yield, duration, and convexity are market-value-weighted averages of the per-bond numbers:

$$y_{\text{port}} = \sum_j w_j y_j, \quad D_{\text{port}} = \sum_j w_j D_{\text{Mod},j}, \quad \text{Cvx}_{\text{port}} = \sum_j w_j \text{Cvx}_j. \quad (30)$$

Portfolio DV01, unlike duration, is not weight-averaged – it’s simply the sum of each holding’s dollar DV01 (DV01 is already a dollar-denominated, additive quantity, so summing directly is correct where a weighted average would double-count the notional):

$$DV01_{\text{port}} = \sum_j DV01_j \times \frac{N_j}{100}. \quad (31)$$

Portfolio KR01 by tenor works the same way – sum each holding’s dollar KR01 at a given key rate tenor, scaled by that holding’s notional.

Worked example. A two-bond portfolio, settled 2025-01-15:

- **Bond A:** 5-year, 4.00% semiannual, \$3,000,000 notional, marked at $y = 4.00\%$ – prices to exactly par, $P = 100.000000$, so $MV_A = \$3,000,000$.
- **Bond B:** 10-year, 4.25% semiannual, \$2,000,000 notional, marked at $y = 4.15\%$ – $P = 100.602130$, so $MV_B = \$2,012,042.59$.

	Market Value	Weight	Mod. Duration	Dollar DV01
Bond A	\$3,000,000.00	59.86%	2.798807	\$840.21
Bond B	\$2,012,042.59	40.14%	6.004548	\$1,208.49
Portfolio	\$5,012,042.59	100%	4.085725	\$2,048.70

The portfolio's weighted duration, $0.598558 \times 2.798807 + 0.401442 \times 6.004548 = 4.085725$, and portfolio DV01, $840.21 + 1208.49 = 2048.70$, match exactly what `Portfolio.weighted_duration()` and `Portfolio.portfolio_dv01()` return for this two-bond example.

13 Modeling Simplifications and Limitations

A few deliberate simplifications keep this platform's scope manageable without misrepresenting what it actually does:

1. **Par yields used directly as spot/discount rates.** A production yield curve pipeline would bootstrap a zero-coupon curve from quoted par yields before using it to discount arbitrary cashflows. This platform skips that step and discounts each cashflow using the (interpolated) par yield at that cashflow's own maturity. For a curve that isn't unusually steep, the resulting pricing and key-rate-duration numbers are directionally correct but not exactly what a bootstrapped zero curve would produce.
2. **Day count convention.** Only Actual/Actual is implemented, which is correct for U.S. Treasury notes and bonds but would need to be extended (e.g. 30/360, Actual/360) to price other fixed income instruments correctly.
3. **Duration uses a simplified time convention.** Macaulay and modified duration (Section 7) measure t_i as whole years from settlement to each cashflow and discount at the flat yield, which is the standard textbook formula but is a slightly different convention from the exact fractional-period exponent used in the pricing engine itself (Equation 3). DV01 and convexity, by contrast, are computed by direct finite difference against the pricing engine, so they do not carry this simplification and serve as a cross-check on the analytical duration figures (see the worked example in Section 8).
4. **Key rate tenor choice is a modeling input**, not something derived from the data. The sample platform uses $\{0.5, 1, 2, 5, 10, 20, 30\}$ years, which is a reasonably standard set but would typically be matched to whatever risk system's key rate buckets a given desk already reports against.